

ChurnGuard AI — Customer Churn Prediction System

Requirements & How to Run the Project

Prepared By	Aryan Sharma
Project	Customer Churn Prediction System (ChurnGuard AI)
Technology	Python · Flask · Scikit-learn
Document Purpose	Environment setup and run instructions for evaluators

1. Project Overview

ChurnGuard AI is an end-to-end machine learning system that predicts telecom customer churn. It trains and compares three classifiers (Random Forest, Gradient Boosting, Logistic Regression) on a 7,043-record Telco Customer Churn dataset, and deploys the best-performing model through a Flask web application with a dashboard, real-time prediction form, analytics visualizations, a dataset explorer, and a methodology page.

2. Requirements

2.1 Software Requirements

Component	Version / Requirement
Operating System	Windows 10/11, macOS 10.15+, or Linux Ubuntu 20.04+
Python	3.9 or higher
pip	Latest version (bundled with Python)
Web Browser	Chrome 90+, Firefox 88+, or Edge 90+
Internet Connection	Required once, to load CDN assets (Plotly.js, Google Fonts)

2.2 Python Package Requirements (requirements.txt)

```
flask==3.0.3
pandas==2.2.2
numpy==1.26.4
scikit-learn==1.5.1
plotly==5.22.0
gunicorn==22.0.0
```

2.3 Hardware Requirements

Resource	Minimum	Recommended
Processor	Intel Core i5 or equivalent	Intel Core i7 or equivalent
RAM	4 GB	8 GB
Storage	500 MB free space	1 GB free space
Display	1366×768	1920×1080

3. Project Structure

```
ChurnGuard-AI/
|-- app.py      # Main Flask application
|-- config.py   # App configuration & credentials
|-- train_model.py # Data generation & model training pipeline
|-- report_config.py # Styling used to generate the PDF report
|-- generate_report.py # Script that builds the PDF project report
|-- report_content.py # Text content used by generate_report.py
|-- requirements.txt # Python package dependencies
|-- templates/  # Jinja2 HTML templates (base, login, dashboard, ...)
|-- static/css/  # style.css -- dark theme design system
|-- data/       # telco_churn.csv, stats.json (auto-generated)
`-- models/     # churn_model.pkl, scaler.pkl, feature_names.pkl (auto-gen)
```

Note: the data/ and models/ folders are created automatically the first time the project is run — they do not need to be created manually.

4. Installation & How to Run

Step 1 — Extract the project files

Unzip the submitted folder (or clone the repository) to a location of your choice on your machine.

Step 2 — Open a terminal in the project folder

```
cd path/to/ChurnGuard-AI
```

Step 3 — (Recommended) Create a virtual environment

```
python -m venv venv
# Windows:
venv\Scripts\activate
# macOS / Linux:
source venv/bin/activate
```

Step 4 — Install dependencies

```
pip install -r requirements.txt
```

Step 5 — Train the model (optional — auto-runs if skipped)

```
python train_model.py
# Generates the synthetic dataset (data/telco_churn.csv),
# trains Random Forest, Gradient Boosting & Logistic Regression,
# and saves the best model + artifacts to models/
```

Step 6 — Start the Flask application

```
python app.py
```

Step 7 — Open the app in your browser

```
http://127.0.0.1:5000
```

If no trained model is found when app.py starts, it automatically invokes train_model.py before launching the server — so Step 5 can safely be skipped on first run.

5. Default Login Credentials

Username	admin
Password	admin123

These are demonstration credentials hardcoded in config.py. For any real deployment, replace them with environment-variable-supplied values and a proper user-management system (see Section 9.3 of the Project Report — Security Considerations).

6. Application Modules

Route	Module	Description
/	Dashboard	KPI cards, churn distribution, contract-type & model comparison charts

/predict	Predict Churn	Real-time individual customer churn prediction with probability score
/analytics	Analytics	Feature importance and churn breakdowns by service/payment/tenure
/dataset	Dataset Explorer	Paginated browsing of all 7,043 customer records
/methodology	Methodology	ML pipeline stages and algorithm formulas

7. Regenerating the PDF Project Report (optional)

The full Project Report PDF included in this submission was generated from report_content.py and report_config.py using generate_report.py. To regenerate it after making changes:

```
pip install reportlab
python generate_report.py
```

8. Troubleshooting

"Model not found" on first run — Normal — the app auto-trains the model. Wait for training to finish (a few seconds) before refreshing the page.

Port 5000 already in use — Stop the process using that port, or edit app.run(port=5000) in app.py to use a different port.

ModuleNotFoundError — Confirm the virtual environment is activated and run pip install -r requirements.txt again.

Charts not loading on Dashboard / Analytics — Ensure you have an active internet connection — Plotly.js is loaded via CDN.